

When Does a Learned World Model Help? A Value-Sufficiency Diagnostic that Predicts World-Model Dependence Across Architectures

Anonymous submission

Abstract

Whether a learned world model (WM) helps a model-based agent is usually debated at the level of *architectures*: latent-consistency (TD-MPC2) versus reconstruction RSSM (Dreamer) versus reward-only value learning. We argue the more predictive question is at the level of *tasks*. We introduce the **Value-sufficiency BottleNeck (VBN)**, a cheap, checkpoint-time probe that measures how much of an agent’s latent the value function actually needs, and show that this single scalar *predicts* whether ablating the world model’s forward-dynamics learning helps or hurts return. Across three DeepMind Control tasks and two independent WM families (Dreamer, TD-MPC2) the same task ordering holds: on ACROBOT-SWINGUP the WM is essential (removing it collapses return by $6.8\times$; $+311$ to $+1458\%$ per seed), on CHEETAH-RUN it helps ($+11\%$), and on WALKER-RUN it is redundant (stripped $\geq$ vanilla). This ordering matches each task’s VBN fingerprint. We also report a clean *negative*: a plan-time “gate” that down-weights the WM’s imagined rollout—the prescriptive fix the diagnostic seems to suggest—does *not* robustly beat trusting the full model ($n=5$; all gate settings inside a 19-point band). Our contribution is therefore a *diagnostic*, not a repaired algorithm: a task property, groundable in the value-equivalence principle and rate-distortion, that tells you *before* you spend the compute whether a learned world model has room to help.

1 Introduction

Model-based reinforcement learning couples a learned *world model* (WM)—a latent dynamics model trained to predict transitions—with a value function and a planner. A large literature asks whether this pays off, and answers by comparing architectures: generative reconstruction models [1], latent-consistency models [2], and value-equivalent models [3, 4]. The empirical picture is mixed, and the mixedness is usually attributed to modeling choices.

We take a different stance. We hold the architecture fixed and ask which *tasks* make the learned dynamics load-bearing. Concretely, we ablate the WM by disabling its forward-dynamics learning objective—in TD-MPC2 by setting the latent-consistency coefficient to zero, in Dreamer by zeroing the dynamics loss—leaving the value function, encoder and

planner otherwise intact. We call the change in return the task’s *WM-dependence*.

Our central finding is that WM-dependence is predictable from a task property we can measure cheaply and *a priori* of the ablation. The property is how compressible the value-relevant information is: if a tiny bottleneck on the value head already recovers most of the return, the value function does not need the full latent, and imposing accurate forward dynamics on that latent is redundant. If the value head needs most of the latent, the forward model’s job of shaping and propagating that information becomes load-bearing. We operationalize this with the Value-sufficiency BottleNeck (VBN).

Contributions.

1. **VBN**, a checkpoint-time probe of value-relevant compressibility, and its three-shape task fingerprint (Sec. 2).
2. A **cross-model, cross-task** result: the same three tasks flip between WM-essential, WM-helpful, and WM-redundant in *both* Dreamer and TD-MPC2, and the ordering matches the VBN fingerprint (Sec. 3).
3. An honest **negative**: the “gated world model” that the diagnostic seems to prescribe does not robustly beat the full model (Sec. 4).
4. A **mechanism**: under planner-driven data collection the WM can actively *hurt* by inflating the planner’s own target variance (Sec. 5).
5. A **theoretical grounding** of VBN in the value-equivalence principle and rate-distortion, and a roadmap toward a training-free analytical criterion (Sec. 6).

2 The Value-sufficiency BottleNeck

Let $z = \phi(o) \in \mathbb{R}^d$ be the agent’s latent and $V_\theta(z)$ its value head. VBN inserts a linear bottleneck of width $D \leq d$ into the value head: $\hat{V}(z) = g_\psi(W_2 \sigma(W_1 z))$ with $W_1 \in \mathbb{R}^{D \times d}$, and re-trains only the small heads while freezing ϕ . Sweeping $D \in \{16, 32, 64, 128\}$ and recording the resulting control return traces a discrete *value rate-distortion* curve: how much return survives when the value function is forced through D dimensions.

Empirically the curve has one of three shapes, which we treat as a task fingerprint:

Task	VBN	Strip-WM effect (Dreamer)	Verdict
ACROBOT	ramp→64	van 412.5 $\gg$ strip 60.8	WM <i>essential</i>
CHEETAH	monotone	van 710 $>$ strip 637	WM <i>helps</i>
WALKER	flat-high	strip 776 $\geq$ van 722	WM <i>redundant</i>

Table 1: The cheap VBN fingerprint predicts the WM-dependence ordering, $n=3$ seeds each. Acrobot: $6.8\times$ gap (+311 to +1458% per seed). Cheetah: +9.7–11.4% across seeds. Walker: null (stripped mildly higher). The same ordering holds in TD-MPC2 (latent-consistency): cheetah WM-load-bearing, walker redundant.

- **monotone** (return keeps rising with D): value needs a large fraction of the latent—low compressibility.
- **flat-high** (already saturated at small D): a tiny bottleneck suffices—high compressibility, value is nearly sufficient in few dimensions.
- **ramp-to- D^*** (rises then plateaus at an intermediate D^*): value needs a specific, sizeable subspace.

In our four-task grid ($n=3$ seeds each), CHEETAH-RUN is monotone, WALKER-RUN is flat-high, ACROBOT-SWINGUP ramps to $D^*=64$, and HOPPER-HOP is flat-high/noisy. VBN costs one short probe-training run per width on an existing checkpoint; it never touches the base policy.

The claim we test is: *low value-compressibility (monotone / ramp) predicts that the learned WM is load-bearing; high compressibility (flat-high) predicts the WM is redundant.*

3 Cross-Model, Cross-Task Result

We ablate the WM in each framework and measure the change in return per task. For Dreamer we report the last-30-episode median return at $\sim 1.1\text{M}$ environment steps; for TD-MPC2 the certified JAX reimplementation matches the official release (cheetah ~ 919 , hopper ~ 581 for the reference implementation; ours is a non-anomalous baseline: hopper within the official seed spread, cheetah -5% , walker -17% , acrobot -23% , with known causes (collection mode, physics backend) under active verification. All TD-MPC2 claims here are within-implementation relative comparisons). The strip ablation removes only the forward-dynamics learning signal.

Table 1 is the core result. Three points deserve emphasis. (i) The ordering is *monotone in the VBN fingerprint*: the more of the latent the value head needs, the more removing the WM hurts. (ii) The ordering is *architecture-invariant*: it reproduces across a reconstruction RSSM (Dreamer) and a latent-consistency model (TD-MPC2), two families that share almost no inductive bias except that they both learn forward dynamics. (iii) The high-dependence anchor is *dramatic but honest*: on acrobot, vanilla return is remarkably stable across seeds (408, 420, 409) while the *stripped* runs are high-variance (26, 56, 100)—sometimes near-total failure, sometimes a partial recovery. The qualitative claim (the WM is essential on acrobot) is unshakable; the magnitude of the collapse varies by seed. We report this rather than hiding the variance.

4 A Clean Negative: The Gate Does Not Help

The diagnostic appears prescriptive: if we can measure, on-line, that a task (or state) is WM-redundant, we should be able to *down-weight* the WM there and win. We implemented exactly this as a plan-time *gate* $g \in [0, 1]$ that scales the planner’s weight on the WM’s imagined rollout, $\text{score} = g \sum_t \gamma^t r_t^{\text{wm}} + \gamma^H V$, so that $g=1$ is the full model and $g \rightarrow 0$ trusts the imagined rollout less.

On CHEETAH-RUN under planner-collection, an early single-seed run suggested a modest win for $g=0.25$ (+5.6%). It did not survive replication. At $n=4-5$ seeds the per-gate means collapse into a 19-point band:

$$\underbrace{701.9}_{g=0.0} \quad \underbrace{688.4}_{g=0.5} \quad \underbrace{686.5}_{g=1.0} \quad \underbrace{682.9}_{g=0.25},$$

while the within-gate seed spread is $\sim 80-100$ points (e.g. $g=0.25$ spans 637–715). No gate value robustly beats the full model; the apparent $n=1$ advantage was seed noise. **The tunable gate is a confirmed negative.**

Why the null is structural, not mysterious. Post-hoc analysis shows the gate as instantiated could hardly have mattered. With horizon $H=3$ and $\gamma=0.99$, the score decomposes as $g \sum_{t<3} \gamma^t r_t + \gamma^3 V \approx 2.97 \bar{r}$ (gated) + $97 \bar{r}$ (terminal): the gated term is $\sim 3\%$ of the trajectory score, so sweeping $g \in [0, 1]$ perturbs the planner objective by at most a few percent—below seed noise by construction. Moreover $g=0$ is *not* WM-free planning: the terminal value $V(z_H)$ is still evaluated on a latent rolled through the learned dynamics, and the value function was trained with the WM regardless; the gate removes imagined *reward*, not imagined *state*. Correct instantiations of the same idea—horizon gating ($H \rightarrow 0$ yields value-greedy planning), per-step uncertainty weighting in the spirit of STEVE [?], or training-time gating—remain untested future work. We keep the negative because it clarifies the contribution: this paper offers a *diagnostic*, not a repaired algorithm, and knowing *whether* a WM will help is separable from—and, on this evidence, easier than—*fixing* one that does not.

5 Mechanism: How the WM Can Hurt

Redundancy alone would predict a null, not a loss. Yet on CHEETAH under *planner-driven* collection (data gathered by the MPPI planner rather than the policy), *removing* the WM *helps* by +45% ($n=9$)—an inversion. We trace it to variance: planner-collection inflates the variance of the planner’s own value target by $\approx 3\times$ (the “H-variance” effect). The imagined rollout, when the value landscape is already nearly sufficient in few dimensions, adds a high-variance term to the plan-time objective that the planner then chases, destabilizing learning. This is the concrete failure mode the diagnostic flags: the WM does not merely fail to help; it can inject target variance where the value function did not need it.

6 Grounding: Compressibility, Value-Equivalence, and an Analytical Criterion

Is “compressibility” well-defined? Yes, information-theoretically—and VBN is a cheap estimator of it, not the exact object. The quantity we care about is the minimal description of state that suffices to represent value. Three standard formalizations coincide here: (1) the *information bottleneck* [5, 6]: $\min_{p(z|s)} I(S; Z)$ s.t. Z is value-sufficient; (2) the *rate-distortion* function $R(D)$ with distortion the value/return error; (3) the *effective dimension* of the value-relevant manifold [8]. VBN’s width sweep is a discrete, coarse sampling of the value rate-distortion curve $R(D)$: the “monotone” fingerprint is a slowly-decaying $R(D)$ (high rate needed), “flat-high” is a sharply-decaying $R(D)$ (low rate suffices). So compressibility is well-defined; VBN is a practical proxy for it, in the same sense a linear probe [7] is a proxy for representational content.

Is VBN novel? Not as an object—and we are explicit about this. Bottlenecking a task head to measure how much representation a function needs is the shared idea behind the information bottleneck [5], probing classifiers [7], and intrinsic-dimension studies [8]. In RL, the *value-equivalence principle* [4] is the direct theoretical cousin: a model need only be accurate on the information that affects value, and MuZero [3] exploits exactly this. State-abstraction theory [9] and bisimulation metrics [10] formalize value-preserving compression of state. Our novelty is not the bottleneck; it is (i) using value-relevant compressibility as a *predictor of when a learned WM helps*, and (ii) validating that prediction *cross-model* on matched tasks. VBN measures the *size* of the value-equivalent subspace; the paper’s claim is that this size governs WM-dependence.

Toward a training-free criterion (answering “can pure analysis decide it?”). VBN still requires a short probe on a trained checkpoint. A genuinely *analytical* criterion—predicting WM-dependence from the task specification alone—appears reachable and is our main open direction. Three candidate proxies, none requiring the full agent, follow from the same logic:

- **Reward-propagation horizon.** The WM helps when credit must travel far. The spectral radius / mixing time of the discounted transition operator applied to the reward field bounds how many steps of lookahead the value needs; long horizons (sparse, underactuated ACROBOT) predict high WM-dependence, short horizons (dense-reward WALKER) predict redundancy—consistent with our data.
- **Controllability / observability structure.** From control theory, underactuated swing-up tasks have ill-conditioned controllability Gramians: reaching the goal requires com-

posing dynamics the policy cannot shortcut, which is exactly where a forward model pays off.

- **Value rate-distortion from the reward geometry.** $R(D)$ for the value can in principle be bounded from the smoothness/Lipschitz structure of r and the transition kernel, giving a closed-form “flat-high vs. monotone” prediction without training.

Whether any of these correlates with the measured WM-dependence gradient across a larger task suite is a clean, falsifiable next experiment. If it does, the diagnostic becomes an *a priori* test—decide whether to pay for a world model before training one.

7 Related Work

Model-based RL with learned latent dynamics: Dreamer [1], TD-MPC2 [2], MuZero [3]. The value-equivalence principle [4] and value-aware model learning motivate models that preserve only value-relevant information; we supply an empirical instrument (VBN) for the *size* of that information and connect it to when the model helps. State abstraction [9] and bisimulation [10] formalize value-preserving compression. The information bottleneck [5, 6], probing [7], and intrinsic dimension [8] are the representational-analysis lineage VBN belongs to. Our tasks are standard DeepMind Control [11].

8 Limitations and Honest Negatives

We report, rather than hide, three tensions. (1) The prescriptive gate fails ($n=5$); the contribution is diagnostic only. (2) On the high-dependence anchor (ACROBOT) the stripped ablation is high-variance across seeds, so WM-dependence *magnitude* is seed-sensitive even though its *sign* is not. (3) VBN requires a short training probe; the training-free criterion of Sec. 6 is conjectural. We also note that HOPPER-VANILLA in Dreamer did not complete on our hardware (repeated initialization OOM), so the cross-model table uses three tasks, not four; the fourth (HOPPER-HOP) contributes only its VBN fingerprint.

9 Conclusion

Whether a learned world model helps is a property of the task’s value-information structure, not of the world-model architecture. A cheap checkpoint-time probe (VBN) measuring value-relevant compressibility predicts the same WM-dependence ordering across two independent world-model families and three tasks: essential on ACROBOT, helpful on CHEETAH, redundant on WALKER. The natural prescriptive fix (a plan-time gate) does not survive replication, sharpening the contribution to a diagnostic. We ground that diagnostic in the value-equivalence principle and rate-distortion, and sketch a training-free analytical criterion—reward-propagation horizon, controllability, and value rate-distortion—that could let

practitioners decide whether a world model has room to help before training one.

References

- [1] D. Hafner, J. Pasukonis, J. Ba, T. Lillicrap. Mastering Diverse Domains through World Models (DreamerV3). *arXiv:2301.04104*, 2023.
- [2] N. Hansen, H. Su, X. Wang. TD-MPC2: Scalable, Robust World Models for Continuous Control. *ICLR*, 2024.
- [3] J. Schrittwieser et al. Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model (MuZero). *Nature*, 2020.
- [4] C. Grimm, A. Barreto, S. Singh, D. Silver. The Value Equivalence Principle for Model-Based Reinforcement Learning. *NeurIPS*, 2020.
- [5] N. Tishby, F. Pereira, W. Bialek. The Information Bottleneck Method. *Allerton*, 1999.
- [6] N. Tishby, N. Zaslavsky. Deep Learning and the Information Bottleneck Principle. *IEEE ITW*, 2015.
- [7] G. Alain, Y. Bengio. Understanding Intermediate Layers Using Linear Classifier Probes. *arXiv:1610.01644*, 2016.
- [8] C. Li, H. Farkhor, R. Liu, J. Yosinski. Measuring the Intrinsic Dimension of Objective Landscapes. *ICLR*, 2018.
- [9] L. Li, T. Walsh, M. Littman. Towards a Unified Theory of State Abstraction for MDPs. *ISAIM*, 2006.
- [10] A. Zhang, R. McAllister, R. Calandra, Y. Gal, S. Levine. Learning Invariant Representations for RL without Reconstruction (DBC). *ICLR*, 2021.
- [11] J. Buckman, D. Hafner, G. Tucker, E. Brevdo, H. Lee. Sample-Efficient RL with Stochastic Ensemble Value Expansion (STEVE). *NeurIPS*, 2018.
- [12] Y. Tassa et al. DeepMind Control Suite. *arXiv:1801.00690*, 2018.